

Optimization-Based Foot Fitting Inside Reconstructed Shoe Meshes

Geometry, SDF, Contact Optimization, and Production Roadmap

Research synthesis for Abhinay Belde

May 20, 2026

Contents

1	Optimization Based Foot Fitting Inside Reconstructed Shoe Meshes	3
1.1	Key Recommendation	3
1.2	Relevant Literature	3
1.3	Shoe Representation and Initialization	4
1.4	Optimization Objective	5
1.5	Implementation and Scaling	7
1.6	Validation, Failure Modes, and Roadmap	8
2	Practical Algorithm Summary	9
2.1	Stage 0: Inputs and Coordinate Convention	9
2.2	Stage 1: Extract Lower Support/Sole Geometry	10
2.3	Stage 2: Infer a Footbed Heightmap	10
2.4	Stage 3: Initialize Foot Pose	11
2.5	Stage 4: Optimize with Staged Losses	11
2.6	Stage 5: Export the Fitted Prior for Neural Training	11
3	PyTorch/trimesh Implementation Details	12
3.1	Mesh Preprocessing	12
3.2	Support-Face Detection	12
3.3	Footprint and Centerline	12
3.4	Differentiable Transform	13
3.5	Grid-Based SDF Query	13
3.6	Robust Penalties	13
3.7	Plantar Contact Interval	14
4	Failure Modes and Automatic Detection	14

25	4.1 Support Extraction Failure	14
26	4.2 Toe-Spring Failure	15
27	4.3 Wrong Yaw	15
28	4.4 Wrong Vertical Seating	15
29	4.5 Boot/Clog/Slide Style Failure	16
30	5 Recommended Outputs Per Shoe	16

1 Optimization Based Foot Fitting Inside Reconstructed Shoe Meshes

1.1 Key Recommendation

The most robust formulation for this setting is not “align a foot to the shoe mesh,” but fit a parametric foot into an inferred pseudo-last or pseudo-cavity derived from the outer shoe mesh. That recommendation is consistent with the footwear literature, which treats the shoe last as the inner volume that matters for fit, and with adjacent vision and graphics literature, which gets robustness by combining model priors, collision exclusion, contact encouragement, and staged optimization rather than relying on raw shape alignment alone.

The footwear papers closest to this problem focus on shoe-last customization, fit evaluation, and 3D ease allowance. The SMPL, scene-contact, and hand-object-contact papers contribute the modern optimization machinery that is useful here. I did not find a widely adopted open paper that takes only an outer shoe mesh and directly fits a parametric foot into it end-to-end; the best solution is a hybrid of these lines of work [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11].

For the canonicalized coordinates, the right geometric anchor is the lower sole or footbed geometry, not whole-shoe PCA. The 3D foot-measurement literature already uses low points near the platform, projection to a horizontal plane, and slice-based boundary extraction to stabilize foot registration. The shoe-last literature explicitly models toe spring, heel height, rearfoot/forefoot zones, and 3D ease allowance rather than assuming a single rigid flat alignment. Biomechanics work on foot-ground contact also suggests that heel and forefoot should be treated separately, because single-shape roll-over abstractions can be too crude [12, 13, 14, 15].

1.2 Relevant Literature

The most relevant footwear-specific papers are these. *Integrated approach to design and manufacture of shoe lasts for orthopaedic use* defines the shoe last as the solid model of the inner volume of the shoe. *Foot measurements from three-dimensional scans* gives a useful registration and slicing mindset for robust geometric measurements. *Shoe-last design innovation for better shoe fitting* is especially important because it explicitly notes that meaningful comparison methods had often been limited to 2D alignment or 3D ICP-like fitness evaluation, which is very close to the gap that appears in the current alignment problem.

A computer-aided design system for foot-feature-based shoe last customization deforms a base last from foot features and visualizes fit through contours. *An analysis and evaluation of fitness for shoe lasts and human feet* uses a fitness function built around ball girth, waist girth, and instep girth, which are repeatedly treated in this literature as key fit measurements. *Customization of shoe last based on 3D design process with adjustable 3D ease allowance* is highly relevant because it models rearfoot and forefoot separately and studies how 3D ease allowance is distributed between foot and last.

Guess your size adds a practical reminder that foot-shape diversity and user preference matter, so a single hard-coded “foot is x% of shoe outer length” rule is too brittle. *Measure2Shape* shows that statistical foot-shape models can be predicted from a small number of measurements with millimeter-scale accuracy, which is useful later if a measurement-driven foot prior is needed [1, 2, 3, 4, 5, 27, 28, 29].

There is also footwear simulation work that is useful conceptually, even if it is often too heavy for large-scale alignment. *Contact model, fit process and foot animation for the virtual footwear simulator of the footwear comfort* and *Human foot modeling towards footwear design model*

foot-footwear interaction and foot deformation for virtual footwear design. The 2022 scoping review on finite-element modeling for footwear concludes that FE methods are useful but still constrained by geometry reconstruction challenges, computational cost, material modeling, realistic loading, and validation burden. That is exactly why a geometry/SDF/contact optimizer is a better first production solution for thousands of shoes than a full FE simulator [18, 19, 20].

For the foot model itself, SUPR-Foot is a strong match to this setup: it provides an articulated foot model with toe joints and learned ground-contact deformations, which gives a principled way to add forefoot or toe articulation instead of using ad hoc nonuniform scaling. Related foot-model work such as FIND, FOUND, and FOCUS is also relevant because it treats foot shape fitting as an optimization problem over a parametric or implicit foot model, even though those papers solve bare-foot reconstruction rather than foot-in-shoe fitting [16, 30, 31, 32].

The most transferable adjacent methods are from human model fitting, human-scene contact, and hand-object contact. SMPLify provides the template for staged optimization with a parametric model plus priors and interpenetration penalties. SMPLify-X adds a better pose prior and a fast interpenetration term. PROX is the closest conceptual analogue to this problem: it extends SMPLify-X with a scene interpenetration term and a contact term that only activates when surfaces are close in distance and orientation. POSA learns a body-centric contact prior that generalizes to new scenes, which is exactly the kind of prior that can later be learned from fitted shoes.

On the self-contact side, explicit contact constraints improve fitting. On the hand-object side, ContactOpt and CPF are especially useful: ContactOpt refines pose using a differentiable contact model, and CPF represents contacts as a spring-like potential field. For collision handling, Local Optimization for Robust Signed Distance Field Collision is a good reference for why SDF-based contact is attractive and how to reason about mesh/SDF collision robustly [6, 7, 8, 9, 17, 10, 11, 21, 22].

1.3 Shoe Representation and Initialization

A robust shoe representation for this case should have three separate pieces:

1. a support footprint on the lower sole,
2. a footbed profile inferred from the lower shoe geometry plus a small latent thickness model,
3. a conservative pseudo-cavity SDF that comes from the outer mesh but is shrunk inward by region-dependent shell-thickness priors.

This is more faithful to shoe-last thinking than trying to interpret the raw outer shell literally as the space available to the foot. The literature on shoe lasts and 3D ease allowance makes the same conceptual distinction between the foot, the last, and the distributed clearance between them [1, 5].

For support-region detection, use only faces or vertices that are both near the bottom-side extreme in the $+Y$ convention and have normals compatible with the outer sole direction. Witana et al. provide a strong precedent for selecting points close to the platform, projecting them, and then using slice boundaries rather than the whole cloud for registration. In this shoe setting, adapt that idea as follows: take a bottom band, project it to the XZ plane, rasterize a footprint occupancy map, close small gaps caused by tread or reconstruction noise, keep the main connected component, and derive a sole-only centerline from the midpoint between the left and right footprint boundaries across x -slices. This answers the “yaw from lower sole geometry

rather than whole-shoe PCA” question. It uses exactly the part of the shoe that should control the foot [12].

A practical sole-only yaw estimator is:

$$\mathcal{F}_{\text{sole}} = \{f : d_b(c_f) \geq q_{0.98}(d_b), \quad n_f \cdot \hat{y}_S \geq \cos \theta_{\text{sole}}\}, \quad (1)$$

where $d_b(p) = p_y$ in the shoe frame, c_f is the face centroid, and n_f is the outward face normal. After projecting $\mathcal{F}_{\text{sole}}$ into XZ , compute left and right footprint boundaries $z_L(x)$ and $z_R(x)$, define the midline

$$m(x) = \frac{1}{2} (z_L(x) + z_R(x)), \quad (2)$$

smooth it with a spline, and initialize yaw from the line joining the heel centroid and the ball-region centroid, not the toe tip and not the shaft. The footwear literature’s emphasis on slice-based measurements and the shoe-last literature’s focus on fit-critical zones both support this footprint-first approach [3, 4, 12].

For toe spring, infer it from the longitudinal footbed profile, not from global pitch. Compute a sole or footbed profile $\bar{B}(x)$ as the median support height in a narrow band around the centerline. Fit a baseline through the rearfoot and midfoot, then treat the persistent front rise above that baseline as the toe-spring curve. The biomechanics literature defines toe spring as the upward forefoot curvature of the shoe and shows that it changes toe-joint mechanics. The shoe-last literature also explicitly states that the foot or last should be adjusted to account for toe spring and heel height. That means the optimizer should not force the entire plantar foot onto one plane. It should allow a separate forefoot hinge or toe articulation [13, 14, 15, 16].

For the pseudo-cavity, the cleanest outer-mesh-only formulation is to create an outer-mesh SDF $D_{\text{out}}(x)$ and then define a conservative cavity SDF

$$D_{\text{cav}}(x) = D_{\text{out}}(x) - T_{\text{shell}}(x; a), \quad (3)$$

where T_{shell} is a low-dimensional shell-thickness model with parameters a . At minimum, a should contain a bottom or insole offset, because that is how unknown sole thickness and footbed height are absorbed. In the improved version, let T_{shell} vary smoothly between rearfoot, midfoot, and forefoot so the footbed can stay approximately parallel to the outsole while still accounting for different heel or rocker constructions.

This is also where the “only outer shoe mesh, no true cavity scan” issue is handled: stop pretending the true cavity is known, and instead optimize inside a conservative cavity proxy with a strong prior on reasonable shell thickness and ease allowance. That is much more production-safe. The shoe-last and 3D-ease literature supports the idea that distributed clearance, not direct foot-surface coincidence, is the quantity to model [1, 5].

1.4 Optimization Objective

If the foot identity is known, the minimum variable set should be

$$\Theta = \{s, \psi, \phi, \rho, t_x, t_y, t_z, q_{\text{mtp}}, a\}, \quad (4)$$

where s is isotropic scale, (ψ, ϕ, ρ) are yaw, pitch, and roll, t is translation, q_{mtp} is a low-dimensional forefoot or toe articulation parameter, and a parameterizes unknown footbed or shell thickness. If the foot identity is not known or only approximately known, add a small number of SUPR shape coefficients β with a strong prior instead of using non-anatomical anisotropic scaling. SUPR is the right model family here because it already includes toe articulation and ground-contact deformations [16].

The objective I would use is:

$$\mathcal{L}(\Theta) = \lambda_{\text{cav}} L_{\text{cav}} + \lambda_{\text{plantar}} L_{\text{plantar}} + \lambda_{\text{seat}} L_{\text{seat}} + \lambda_{\text{axis}} L_{\text{axis}} + \lambda_{\text{girth}} L_{\text{girth}} + \lambda_{\text{toe}} L_{\text{toe}} + \lambda_{\text{prior}} L_{\text{prior}} + \lambda_{\text{sym}} L_{\text{sym}}. \quad (5)$$

This structure is directly inspired by SMPLify/SMPLify-X style parametric fitting, PROX-style exclusion plus contact, and ContactOpt/CPF style soft-contact energies, while the footwear-specific parts come from shoe-last ease allowance and girth-based fit work [4, 6, 7, 8, 10, 11].

A good containment term is a one-sided barrier on the cavity SDF:

$$L_{\text{cav}} = \frac{1}{|S|} \sum_{i \in S} \phi_+(m_{\text{wall}} - D_{\text{cav}}(v_i(\Theta))), \quad (6)$$

where S is a dense sample of foot surface points, m_{wall} is the desired minimum wall clearance, and $\phi_+(u)$ is a softplus or Huber barrier. This is the direct geometry/SDF/collision optimization term. Use dense samples, not only mesh vertices, if the foot mesh is coarse. Smoothed SDF-based collision is a natural match to PROX-like exclusion terms and to the SDF collision literature [8, 21].

For footbed contact, do not force the entire plantar surface onto the support surface. Define anatomical plantar masks such as heel pad, lateral and medial metatarsal heads, hallux pad, mid-arch, and toe pads. If $B(x, z; a)$ is the inferred footbed height in the +Y-bottom convention, define plantar clearance

$$c_i(\Theta) = B(v_i^x, v_i^z; a) - v_i^y. \quad (7)$$

Then use an interval loss rather than a zero-distance loss:

$$L_{\text{plantar}} = \frac{1}{|\mathcal{P}|} \sum_{i \in \mathcal{P}} w_i [\phi_+(\ell_i - c_i) + \phi_+(c_i - u_i)]. \quad (8)$$

Here ℓ_i and u_i are region-specific lower and upper bounds. Heel and met-head regions should have tighter target ranges, while the arch and toe regions should allow more float. This follows the shoe-last/ease literature and the biomechanics literature: a comfortable fit is about distributed ease and blocking, not globally zero clearance, and separate heel/forefoot mechanics matter [5, 14, 15].

For heel seating and longitudinal scale, explicitly optimize the heel gap and toe gap. Let $x_{\text{heel}}^{\text{wall}}$ and $x_{\text{toe}}^{\text{wall}}$ be inferred internal heel and toe planes from the pseudo-cavity, and let $x_{\text{min}}^{\text{heel}}$ and $x_{\text{max}}^{\text{toe}}$ be the posterior heel and anterior toe extents of the transformed foot. Then

$$g_h = x_{\text{min}}^{\text{heel}} - x_{\text{heel}}^{\text{wall}}, \quad g_t = x_{\text{toe}}^{\text{wall}} - x_{\text{max}}^{\text{toe}}. \quad (9)$$

Use interval or soft target losses on both. This is how scale should really be estimated: not from a fixed fraction of outer length, but from heel seating, toe allowance, and cross-sectional fit together. The footwear fit literature repeatedly centers fit on the ball, waist, and instep regions, and the size-recommendation literature shows foot-shape variation is too large for a single hard-coded scale rule [4, 27].

For yaw, pitch, and roll, use separate terms. Yaw should stay close to the sole-derived centerline. Pitch should be weakly attracted to the shoe's toe-spring profile through q_{mtp} rather than through global pitch alone. Roll should be weakly regularized toward the local support plane inferred from heel and forefoot slices. A useful alignment term is

$$L_{\text{axis}} = \frac{1}{K} \sum_{k=1}^K \rho(z_{\text{axis}}^{\text{foot}}(x_k) - m(x_k)) + \rho(\psi - \psi_{\text{sole}}), \quad (10)$$

where $z_{\text{axis}}^{\text{foot}}(x_k)$ is the center of the foot section at slice x_k , $m(x)$ is the sole-only centerline, and ρ is a robust penalty. For toe spring, either use SUPR toe joints or a smooth, blended forefoot transform around the metatarsophalangeal axis; do not try to absorb everything into global pitch. That is the failure mode that makes fits look too flat or too high in the front [13, 14, 15, 16].

The foot SDF is also worth using. A symmetric narrow-band term helps detect collisions that a surface-only term can miss:

$$L_{\text{sym}} = \frac{1}{|Q|} \sum_{q \in Q} \phi_+ (m_{\text{sym}} - D_{\text{foot}}(T_{\Theta}^{-1}(q))), \quad (11)$$

where Q is a sample of pseudo-cavity boundary points, D_{foot} is the foot SDF in raw SUPR coordinates, and T_{Θ}^{-1} maps shoe-frame points back into foot-frame coordinates. This is a direct way to exploit the existing SDF and is in the spirit of mesh/SDF collision handling [21, 22].

1.5 Implementation and Scaling

In Python, use `trimesh` for mesh cleaning, slicing, footprint extraction, and CPU-side sanity checks, but run the final objective in PyTorch against precomputed grids. `trimesh.intersections.mesh_mult` is a convenient way to slice many x -planes, and `trimesh.proximity.signed_distance` is useful for debugging. Note the library’s sign convention: it reports positive distance inside the mesh and negative distance outside, which is the opposite of many SDF conventions. Normalize that sign once and unit-test it. Trimesh also exposes ICP, but its own documentation notes that ICP only behaves reasonably when the initialization is already roughly correct, which is exactly why ICP should remain a diagnostic tool here, not the main fitter [23].

The practical production path is to precompute a shoe-footbed grid and a pseudo-cavity SDF grid, then query them on the GPU with `torch.nn.functional.grid_sample`. PyTorch’s `grid_sample` supports both 2D and volumetric sampling, which makes it a good fit for this kind of differentiable geometry objective. For optimization, use Adam first to get into the right basin, then switch to LBFGS for cleanup; PyTorch’s LBFGS reevaluates the objective through a closure, which is exactly what is needed once the candidate is close [24, 25, 26].

A practical optimizer skeleton looks like this:

```
# Precompute from shoe mesh
support = detect_support_faces(shoe_mesh) # lower sole only
footprint = rasterize_footprint(support)
centerline = fit_centerline_from_footprint(footprint)
footbed = build_footbed_heightmap(shoe_mesh, support)
cavity_sdf = build_pseudocavity_sdf(shoe_mesh, style_prior)

# Parameters
params = init_from_support(centerline, footbed, cavity_sdf, supr_foot)

# Coarse stage
adam = torch.optim.Adam(params.trainables(), lr=1e-2)
for _ in range(200):
    adam.zero_grad()
    loss = objective(params, cavity_sdf, footbed, centerline, foot_sdf)
    loss.backward()
    adam.step()

# Fine stage
lbfgs = torch.optim.LBFGS(
```

```

246     params.trainables(), max_iter=40, line_search_fn="strong_wolfe"
247 )
248
249 def closure():
250     lbfgs.zero_grad()
251     loss = objective(params, cavity_sdf, footbed, centerline, foot_sdf)
252     loss.backward()
253     return loss
254
255 lbfgs.step(closure)
256

```

257 That coarse-to-fine schedule closely mirrors the broader model-fitting literature, but the impor-
258 tant domain-specific difference is that initialization should come from the support representation
259 rather than raw PCA or a weak ankle-opening guess [6, 8, 10].

260 To make the system robust over thousands of shoes, add three production features. First, run
261 multi-start from a small pool of seeds that vary yaw, scale, forefoot hinge, and heel-seat offset
262 around the sole-derived initialization; this catches local minima. Second, compute simple style
263 descriptors from the outer mesh: shaft height, back height, opening confidence, sole curvature,
264 heel-to-forefoot drop, and footprint aspect ratio. Use them either to hard-route shoes into
265 rough families such as low shoe, boot, clog or mule, and slide, or to regress the shell-thickness
266 parameters a . Third, once enough successful fits are available, learn a POSA-like prior for foot-
267 shoe contact and a SMPLify-like initializer that predicts Θ from shoe descriptors, then keep the
268 optimizer as the final refinement layer. That gives a fitted pose and alignment prior that can
269 be reused during neural shoe-mesh training [6, 8, 9].

270 1.6 Validation, Failure Modes, and Roadmap

271 The most useful debug outputs are not just the final overlay. Generate a sole-only footprint
272 plot with the extracted centerline overlaid; a longitudinal footbed profile showing heel, mid-
273 foot, forefoot, and toe-spring rise; cross-sections at heel, waist, ball, instep, and toe; a plantar
274 clearance heatmap on the foot; a penetration heatmap on foot and shoe; and a contour-style fit
275 visualization similar in spirit to the contour figure used in shoe-last customization work.

276 Quantitatively, log at least these metrics per fit: minimum cavity clearance, heel gap, toe
277 gap, centerline misalignment angle, forefoot pitch relative to the extracted toe-spring angle,
278 ball/waist/instep slack, plantar-support coverage on heel and met-head masks, the fraction of
279 plantar vertices that are floating beyond tolerance, and multi-start parameter spread. The
280 emphasis on ball/waist/instep measurements is directly supported by the shoe-fit literature [3,
281 4].

282 The main failure modes are predictable. Toe spring errors show up as good heel seating but
283 excessive forefoot penetration or excessive toe float. Heel seating errors show up as a large heel
284 gap combined with seemingly good toe fit. Yaw errors show up as one sidewall colliding while
285 the opposite side has too much slack, even though length looks correct. Scale errors show up in
286 ball or instep overfill even when toe and heel gaps look plausible. Boot failures usually mean
287 the shaft volume is still influencing the fit, which can be detected when a fit looks acceptable
288 in the upper but poor at the outsole. Slide or clog failures usually come from over-using dorsal
289 or heel-counter terms that should have been disabled or weakened. High disagreement between
290 multi-start solutions is also a strong automatic QC signal.

291 The broader contact-fitting literature and the footwear literature both imply that contact-aware
292 constraints help only when the contact surfaces themselves are well identified; if the support
293 map is wrong, the optimizer will confidently solve the wrong problem [5, 8, 10, 15].

Simple baseline. Start with a rigid or similarity fit of a fixed SUPR foot that optimizes only $\{s, \psi, \phi, \rho, t\}$ against a sole-derived centerline, heel gap, toe gap, ball-width slack, and a constant-offset footbed. Do not use whole-shoe PCA at all. This version already addresses the biggest problems: wrong yaw, wrong vertical seating, and wrong global scale. It will also be easy to validate and fast enough to batch over a large catalog [12].

Improved optimizer. Add a forefoot hinge or SUPR toe articulation, a low-dimensional shell-thickness model a , dense pseudo-cavity SDF containment, and the symmetric foot-SDF term. Partition the plantar mask into heel, arch, met-head, hallux, and toes, and use interval-based support rather than hard contact. This is the version expected to handle toe spring, clogs, slides, rocker soles, and many boots much better than the current heuristic pipeline. It is also the right place to introduce style-conditioned priors and multi-start fitting [10, 14, 16, 21].

Production-scale version. After a few thousand successful fits, learn a lightweight predictor from shoe descriptors to the initial parameter distribution (μ, Σ) , the shell-thickness prior a , and a foot-shoe contact prior over plantar vertices. Then use the optimizer only as a refinement and QC stage. In parallel, use the fitted transforms and contact maps as priors during neural shoe reconstruction or canonicalization training, exactly the way SMPLify-style optimization has often been used to bootstrap learned regressors, and the way POSA turns fitted interactions into a reusable contact prior. This is the version most likely to be robust at scale without hand-tuning every style family [6, 9].

2 Practical Algorithm Summary

This section rewrites the method in an implementation-oriented way so it is easier to turn into code.

2.1 Stage 0: Inputs and Coordinate Convention

Inputs:

1. Canonicalized shoe mesh $M_S = (V_S, F_S)$ with $+X$ heel-to-toe, $+Y$ bottom/sole side, $-Y$ opening/up side, and $+Z$ width direction.
2. SUPR foot mesh $M_F(\beta, q)$ or a static SUPR foot mesh in raw SUPR coordinates.
3. Foot SDF D_{foot} in raw SUPR coordinates.
4. Optional foot anatomical masks: heel, arch, metatarsal heads, hallux, toes, dorsal foot, ankle loop.

Outputs:

1. Similarity transform (s, R, t) , where $R = R_y(\psi)R_z(\phi)R_x(\rho)$ or the equivalent for the shoe frame.
2. Optional toe/forefoot articulation q_{mtp} .
3. Optional shell/footbed offset parameters a .

4. Per-vertex or per-sample diagnostics: cavity clearance, plantar clearance, support mask, sidewall penetration, heel/toe gaps.

The key engineering principle is: estimate the pose from the support geometry, then use the whole outer mesh only as a soft, conservative constraint.

2.2 Stage 1: Extract Lower Support/Sole Geometry

Use the shoe’s +Y-bottom convention. If canonicalization is correct, bottom-side faces should have large y values and outward normals that roughly point along +Y. Select a bottom band and apply a normal filter:

$$\mathcal{F}_{\text{sole}} = \{f : y(c_f) \geq q_\tau(y(c_f)), \quad n_f \cdot \hat{y}_S \geq \cos \theta\}, \quad (12)$$

where q_τ can start near 0.95 to 0.98, and θ can start around 45° . For noisy GShell meshes, use a wider bottom band and then keep the largest connected component after projection.

Then rasterize (x, z) of the selected faces onto a 2D grid. Perform binary closing and hole filling. From this footprint, compute:

- heel region: lowest x or rear 10–15% of footprint length,
- ball region: roughly 60–75% of footprint length from heel,
- toe region: front 10–15%,
- boundaries $z_L(x)$ and $z_R(x)$,
- centerline $m(x) = \frac{1}{2}(z_L(x) + z_R(x))$,
- sole yaw ψ_{sole} from heel-to-ball or heel-to-midfoot line.

This avoids upper/shaft bias because boots and high uppers can dominate whole-mesh PCA, but they should not decide the foot’s centerline.

2.3 Stage 2: Infer a Footbed Heightmap

Because only the outer shoe mesh is available, the true insole or inner cavity is unknown. Use a conservative proxy:

$$B(x, z; a) = B_{\text{outer}}(x, z) - T_{\text{bottom}}(x, z; a), \quad (13)$$

where $B_{\text{outer}}(x, z)$ is the observed bottom/sole-side height and T_{bottom} is an unknown thickness or offset. In the +Y-bottom convention, be consistent about signs: if larger y is lower, then moving from outsole toward the footbed generally decreases y . The equation above is conceptual; implement it after confirming the sign with a small visualization.

Start with a constant T_{bottom} . Then upgrade to:

$$T_{\text{bottom}}(x, z; a) = a_0 + a_1 r_{\text{heel}}(x) + a_2 r_{\text{mid}}(x) + a_3 r_{\text{fore}}(x), \quad (14)$$

where r_{heel} , r_{mid} , and r_{fore} are smooth radial-basis or spline weights along shoe length. This gives the optimizer enough flexibility for heel lift, rocker, and toe spring without inventing arbitrary deformations.

2.4 Stage 3: Initialize Foot Pose

Initialize scale from inner length, not outer length:

$$s_0 = \frac{L_{\text{pseudo-inner}} - g_{\text{toe,target}} - g_{\text{heel,target}}}{L_{\text{foot}}}. \quad (15)$$

Initialize yaw from the lower sole centerline ψ_{sole} . Initialize vertical translation so the plantar heel and met-head masks have small positive clearance above the inferred footbed. Initialize pitch from the rearfoot-midfoot support plane, not from the entire shoe. Initialize roll from a robust plane fit to the rearfoot and forefoot support points.

Run 5–20 multi-start candidates:

$$\psi = \psi_{\text{sole}} + \delta_\psi, \quad s = s_0(1 + \delta_s), \quad q_{\text{mtp}} = \delta_q, \quad (16)$$

where $\delta_\psi \in \{-5^\circ, 0, 5^\circ\}$, $\delta_s \in \{-0.03, 0, 0.03\}$, and δ_q samples plausible toe-spring angles. Keep the best candidate by objective value and QC metrics.

2.5 Stage 4: Optimize with Staged Losses

A stable staged schedule is usually better than activating all losses at full strength from the start.

Stage A: coarse similarity fit. Optimize s, ψ, t_x, t_y, t_z with centerline, heel gap, toe gap, and plantar interval losses. Keep pitch/roll weakly regularized.

Stage B: 6-DoF fit. Add pitch/roll and pseudo-cavity containment. Turn on sidewall/girth losses.

Stage C: forefoot and shell refinement. Add q_{mtp} , shell thickness a , symmetric foot SDF term, and stronger contact-region losses.

Stage D: final cleanup. Use LBFGS for 20–50 iterations with all losses active and smaller robust margins.

2.6 Stage 5: Export the Fitted Prior for Neural Training

For each fitted shoe, save:

- `foot_to_shoe_transform.json`,
- `fitted_foot_mesh.obj`,
- `plantar_contact_mask.npy`,
- `cavity_clearance.npy`,
- `footbed_heightmap.npy`,
- `fit_metrics.json`,
- visualization images for QC.

During neural mesh training, use these outputs as priors. For example, use the foot mesh as a weak occupancy exclusion prior, use plantar contact maps to regularize open-surface reconstruction around the footbed, and use fitted transforms to canonicalize shoe/foot relations across the dataset.

3 PyTorch/trimesh Implementation Details

3.1 Mesh Preprocessing

Use `trimesh` for cleaning and topological sanity:

```
import trimesh

shoe = trimesh.load(shoe_path, force="mesh")
shoe.remove_duplicate_faces()
shoe.remove_degenerate_faces()
shoe.remove_unreferenced_vertices()
shoe.merge_vertices()

# Optional smoothing only for support extraction, not for final geometry.
shoe_for_support = shoe.copy()
```

Do not over-smooth the actual mesh used for SDF construction, because smoothing can erase useful boundaries. Smooth only the extracted support signal or heightmap.

3.2 Support-Face Detection

```
import numpy as np

def detect_support_faces(mesh, q=0.97, normal_cos=0.5):
    V = mesh.vertices
    F = mesh.faces
    centroids = V[F].mean(axis=1)
    normals = mesh.face_normals

    y_thresh = np.quantile(centroids[:, 1], q) # +Y is bottom
    bottom_band = centroids[:, 1] >= y_thresh
    normal_ok = normals[:, 1] >= normal_cos
    idx = np.where(bottom_band & normal_ok)[0]
    return idx
```

In practice, use this as a first pass. If too few faces are selected, relax the normal threshold and widen the bottom band. If too many faces are selected in a boot or clog, use connected components in projected XZ footprint and keep the largest sole-like component.

3.3 Footprint and Centerline

```
from scipy.ndimage import binary_closing, binary_fill_holes
from scipy.interpolate import UnivariateSpline

# Pseudocode only
def footprint_centerline(points_xz, grid_res=256):
    occ, x_edges, z_edges = rasterize(points_xz, grid_res)
    occ = binary_closing(occ, iterations=2)
    occ = binary_fill_holes(occ)
    occ = keep_largest_component(occ)

    xs = []
    mids = []
    widths = []
```

```

449     for ix in range(occ.shape[0]):
450         cols = np.where(occ[ix])[0]
451         if len(cols) < 3:
452             continue
453         z_left = z_edges[cols.min()]
454         z_right = z_edges[cols.max()]
455         x_mid = 0.5 * (x_edges[ix] + x_edges[ix + 1])
456         xs.append(x_mid)
457         mids.append(0.5 * (z_left + z_right))
458         widths.append(z_right - z_left)
459
460     spline = UnivariateSpline(xs, mids, s=0.001 * len(xs))
461     return spline, np.asarray(xs), np.asarray(widths)

```

463 Use the spline only over the stable central part of the footprint. Do not trust very front toe
464 slices or rear heel slices if they are too narrow or noisy.

465 3.4 Differentiable Transform

466 Represent scale, Euler angles, and translation as PyTorch tensors. Keep the rotation param-
467 eterization simple at first, because only small pitch/roll changes are needed. Later, switch to
468 quaternions or Lie algebra if needed.

```

469 import torch
470
471
472 def transform_points(V, log_s, yaw, pitch, roll, t):
473     s = torch.exp(log_s)
474     R = euler_to_matrix(yaw, pitch, roll) # define for your axis convention
475     return s * (V @ R.T) + t

```

477 Use `log_s` rather than `s` directly so the scale stays positive. Put tight priors on all angles and
478 scale.

479 3.5 Grid-Based SDF Query

480 Build SDF grids in shoe coordinates and sample them using `torch.nn.functional.grid_sample`.
481 Normalize query coordinates into $[-1, 1]^3$ before sampling.

```

482 import torch.nn.functional as F
483
484
485 def sample_grid_3d(grid, xyz, bbox_min, bbox_max):
486     # grid: [1, 1, D, H, W]
487     # xyz: [N, 3] in world/shoe coordinates
488     uvw = 2.0 * (xyz - bbox_min) / (bbox_max - bbox_min) - 1.0
489     # grid_sample expects [Nbatch, Dout, Hout, Wout, 3]
490     query = uvw.view(1, -1, 1, 1, 3)
491     val = F.grid_sample(grid, query, align_corners=True, mode="bilinear")
492     return val.view(-1)

```

494 Unit-test SDF sign conventions with one point clearly outside the shoe and one point clearly
495 inside the outer mesh. Then lock the convention in code comments.

496 3.6 Robust Penalties

497 Use softplus barriers or Huberized hinge losses:

```

498
499 def smooth_hinge(x, beta=20.0):
500     # approximates max(x, 0)
501     return torch.nn.functional.softplus(beta * x) / beta
502
503 def huber(x, delta=0.005):
504     ax = x.abs()
505     return torch.where(ax < delta, 0.5 * x * x / delta, ax - 0.5 * delta)
506

```

507 For example, containment can be implemented as:

```

508
509 clearance = sample_grid_3d(cavity_sdf_grid, foot_samples, bbox_min, bbox_max)
510 loss_cav = smooth_hinge(m_wall - clearance).mean()
511

```

512 3.7 Plantar Contact Interval

```

513
514 def plantar_interval_loss(foot_pts, plantar_idx, footbed_grid, lower, upper, weights):
515     P = foot_pts[plantar_idx]
516     B = sample_heightmap_2d(footbed_grid, P[:, [0, 2]])
517     c = B - P[:, 1] # confirm sign for +Y-bottom convention
518     lo = lower[plantar_idx]
519     hi = upper[plantar_idx]
520     w = weights[plantar_idx]
521     return (w * (smooth_hinge(lo - c) + smooth_hinge(c - hi))).mean()
522

```

523 The sign of c depends on whether the heightmap stores the outsole, insole, or bottom-side
524 coordinate. Before running thousands of shoes, plot c for one known good manual alignment
525 and verify that contact regions have the expected small positive clearance.

526 4 Failure Modes and Automatic Detection

527 4.1 Support Extraction Failure

528 Symptoms: centerline jumps, footprint is split into multiple large components, yaw is unstable
529 across small threshold changes, or the selected support area is extremely small.

530 Detection metrics:

- 531 • support area divided by shoe footprint area,
- 532 • number of connected components,
- 533 • centerline curvature outliers,
- 534 • yaw difference between thresholds $q = 0.95, 0.97, 0.98$,
- 535 • width profile discontinuities.

536 Automatic response: widen bottom band, relax normal filter, apply projection-space morphol-
537 ogy, or fall back to lower 20% mesh slices rather than face normals.

4.2 Toe-Spring Failure

Symptoms: heel looks seated but forefoot penetrates the footbed, or the toe floats far above the toe box/footbed.

Detection metrics:

- forefoot plantar clearance mean and max,
- toe-pad floating fraction,
- estimated toe-spring angle mismatch,
- high loss on q_{mtp} prior.

Automatic response: enable forefoot hinge, increase toe-spring range, or classify the shoe as rocker/raised-toe style.

4.3 Wrong Yaw

Symptoms: one medial/lateral side collides and the opposite side has large slack; length and vertical placement may look correct.

Detection metrics:

- left/right sidewall slack imbalance,
- large L_{axis} ,
- multi-start yaw disagreement,
- width profile overfill on only one side.

Automatic response: rerun yaw multi-start, use heel-to-ball centerline instead of full centerline, or downweight noisy toe slices.

4.4 Wrong Vertical Seating

Symptoms: foot floats above the footbed or penetrates below it.

Detection metrics:

- plantar support coverage,
- heel clearance median,
- met-head clearance median,
- fraction of plantar samples outside interval.

Automatic response: adjust a_0 , optimize vertical translation first, and make footbed loss dominate before turning on full SDF containment.

4.5 Boot/Clog/Slide Style Failure

Symptoms: boot shaft biases cavity/SDF, clogs lack a heel counter, slides have no full upper, or dorsal contact terms pull the foot into nonsensical places.

Detection metrics:

- shaft height ratio,
- heel counter confidence,
- upper coverage over instep,
- opening confidence,
- dorsal containment confidence.

Automatic response: style-route losses. For slides, reduce heel-counter and dorsal containment terms. For boots, downweight shaft geometry and rely more on lower footbed and forefoot/heel seating. For clogs, use weaker heel seating and stronger footbed/forefoot constraints.

5 Recommended Outputs Per Shoe

For each shoe, generate:

1. `overlay_side.png`: shoe plus fitted foot from lateral side.
2. `overlay_top.png`: shoe plus fitted foot from opening/top side.
3. `footprint_centerline.png`: projected sole footprint and extracted centerline.
4. `footbed_profile.png`: longitudinal footbed profile and toe-spring estimate.
5. `cross_sections.png`: heel, waist, ball, instep, and toe slices.
6. `plantar_clearance_heatmap.png`: color-coded foot plantar surface.
7. `penetration_heatmap.png`: foot and shoe collision heatmap.
8. `fit_metrics.json`: scalar QC metrics.
9. `fit_status.json`: pass/warn/fail and failure reason.

A production run should not rely on visual inspection for every shoe, but it should save enough visualizations that failed cases are easy to diagnose.

References

- [1] H. P. Bao et al., “Integrated approach to design and manufacture of shoe lasts for orthopaedic use.” ScienceDirect. <https://www.sciencedirect.com/science/article/abs/pii/S0360835294900744>
- [2] A. Luximon et al., “Shoe-last design innovation for better shoe fitting.” ScienceDirect. <https://www.sciencedirect.com/science/article/abs/pii/S0166361509001365>

- 598 [3] “A computer-aided design system for foot-feature-based shoe last customization.” Springer.
599 <https://link.springer.com/article/10.1007/s00170-009-2087-7>
- 600 [4] “An analysis and evaluation of fitness for shoe lasts and human feet.” ResearchGate /
601 source listing. https://www.researchgate.net/publication/222023003_An_analysis_and_evaluation_of_fitness_for_shoe_last_and_human_feet
602
- 603 [5] “Customization of shoe last based on 3D design process with adjustable 3D ease allowance.”
604 Springer. <https://link.springer.com/article/10.1007/s00170-022-10427-5>
- 605 [6] F. Bogo et al., “SMPLify: 3D Human Pose and Shape from a Single Image.” Project page.
606 <https://smplify.is.tue.mpg.de/>
- 607 [7] G. Pavlakos et al., “Expressive Body Capture: 3D Hands, Face, and Body from a Single
608 Image” / SMPLify-X. arXiv. <https://arxiv.org/abs/1904.05866>
- 609 [8] M. Hassan et al., “Resolving 3D Human Pose Ambiguities with 3D Scene Constraints” /
610 PROX. Project page. <https://prox.is.tue.mpg.de/>
- 611 [9] M. Hassan et al., “POSA: Populating 3D Scenes by Learning Human-Scene Interaction.”
612 Project page. <https://posa.is.tue.mpg.de/>
- 613 [10] P. Grady et al., “ContactOpt: Optimizing Contact to Improve Grasps.” Project page.
614 <https://www.pgrady.net/contactopt/>
- 615 [11] “CPF: Learning a Contact Potential Field for Modeling the Hand-Object Interaction.”
616 arXiv. <https://arxiv.org/abs/2012.00924>
- 617 [12] C. P. Witana et al., “Foot measurements from three-dimensional scans.” ScienceDirect.
618 <https://www.sciencedirect.com/science/article/abs/pii/S0169814106001260>
- 619 [13] “3D Foot Scan to Custom Shoe Last.” CAD Journal PDF. https://www.cad-journal.net/files/vol_2/CAD_2%281-4%29_2005_11-18.pdf
620
- 621 [14] “Toe springs affect foot biomechanics.” Scientific Reports / Nature. <https://www.nature.com/articles/s41598-020-71247-9>
622
- 623 [15] “Foot-ground contact / roll-over and foot mechanics.” Frontiers in Neurorobotics.
624 <https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2019.00062/full>
625
- 626 [16] SUPR-Foot: articulated foot model with toe joints and contact deformations. Project page.
627 <https://supr.is.tue.mpg.de/>
- 628 [17] “On Self-Contact and Human Pose.” CVPR 2021 paper. https://openaccess.thecvf.com/content/CVPR2021/papers/Muller_On_Self-Contact_and_Human_Pose_CVPR_2021_paper.pdf
629
630
- 631 [18] Y. Song et al., “Finite element modelling for footwear design and evaluation: A scop-
632 ing review.” ScienceDirect. <https://www.sciencedirect.com/science/article/pii/S2405844022022289>
633
- 634 [19] M. J. Ruperez et al., “Contact model, fit process and foot animation for the virtual simula-
635 tor of the footwear comfort.” ScienceDirect. <https://www.sciencedirect.com/science/article/abs/pii/S0010448509002346>
636
- 637 [20] Y. M. Tang et al., “Human foot modeling towards footwear design.” ScienceDirect. <https://www.sciencedirect.com/science/article/abs/pii/S001044851100193X>
638

- 639 [21] “Local Optimization for Robust Signed Distance Field Collision.” ACM Digital Library.
640 <https://dl.acm.org/doi/10.1145/3384538>
- 641 [22] M. Macklin, “SDF Contact.” PDF. <https://mmacklin.com/sdfcontact.pdf>
- 642 [23] Trimesh documentation: intersections, proximity, and registration. [https://trimesh.](https://trimesh.org/)
643 [org/; https://trimesh.org/trimesh.intersections.html](https://trimesh.org/trimesh.intersections.html); [https://trimesh.org/](https://trimesh.org/trimesh.proximity.html)
644 [trimesh.proximity.html](https://trimesh.org/trimesh.registration.html); <https://trimesh.org/trimesh.registration.html>
- 645 [24] PyTorch documentation: `torch.nn.functional.grid_sample`. [https://docs.pytorch.](https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.grid_sample.html)
646 [org/docs/stable/generated/torch.nn.functional.grid_sample.html](https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.grid_sample.html)
- 647 [25] PyTorch documentation: `torch.optim.Adam`. [https://docs.pytorch.org/docs/](https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html)
648 [stable/generated/torch.optim.Adam.html](https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html)
- 649 [26] PyTorch documentation: `torch.optim.LBFGS`. [https://docs.pytorch.org/docs/](https://docs.pytorch.org/docs/stable/generated/torch.optim.LBFGS.html)
650 [stable/generated/torch.optim.LBFGS.html](https://docs.pytorch.org/docs/stable/generated/torch.optim.LBFGS.html)
- 651 [27] S. Huang et al., “Guess your size: A hybrid model for footwear size recommen-
652 dation.” ScienceDirect. [https://www.sciencedirect.com/science/article/abs/pii/](https://www.sciencedirect.com/science/article/abs/pii/S1474034617302367)
653 [S1474034617302367](https://www.sciencedirect.com/science/article/abs/pii/S1474034617302367)
- 654 [28] Z. Zhu et al., “Measure2Shape / A novel footwear customisation framework utilising 3D an-
655 thropometric measurements.” ScienceDirect. [https://www.sciencedirect.com/science/](https://www.sciencedirect.com/science/article/abs/pii/S0166361525000223)
656 [article/abs/pii/S0166361525000223](https://www.sciencedirect.com/science/article/abs/pii/S0166361525000223)
- 657 [29] “A feature-based shoe-last design and automatic model construction.” CAD Journal. [https:](https://cad-journal.net/files/vol_22/CAD_22%284%29_2025_600-615.pdf)
658 [//cad-journal.net/files/vol_22/CAD_22%284%29_2025_600-615.pdf](https://cad-journal.net/files/vol_22/CAD_22%284%29_2025_600-615.pdf)
- 659 [30] “FIND: Parametric foot model / foot fitting related work.” arXiv. [https://arxiv.org/](https://arxiv.org/abs/2210.12241)
660 [abs/2210.12241](https://arxiv.org/abs/2210.12241)
- 661 [31] “FOUND: Foot optimization / foot reconstruction related work.” arXiv. [https://arxiv.](https://arxiv.org/abs/2310.18279)
662 [org/abs/2310.18279](https://arxiv.org/abs/2310.18279)
- 663 [32] “FOCUS: Foot reconstruction / foot model related work.” arXiv. [https://arxiv.org/](https://arxiv.org/abs/2502.06367)
664 [abs/2502.06367](https://arxiv.org/abs/2502.06367)
- 665 [33] L. Muller et al., “On Self-Contact and Human Pose.” CVPR 2021. [https://arxiv.org/](https://arxiv.org/abs/2104.03176)
666 [abs/2104.03176](https://arxiv.org/abs/2104.03176)